

GPT-1

Improving Language Understanding by Generative Pre-Training



- **텍스트 분류:**

문장이 긍정이야? 부정이야? 스포츠 기사야? 정치 기사야? 이런 라벨을 붙이는 작업

- **시퀀스 라벨링:**

문장 안의 각 단어에 라벨을 붙이는 작업

예: 사람 이름, 장소, 날짜 같은 걸 찾는 Named Entity Recognition (NER)

1. Abstract

문제

자연어 이해(NLU)는 텍스트 간 논리 관계 판단, 질문응답, 의미 유사도 계산, 문서 분류 등 다양한 작업 포함

but... 라벨링 된 데이터 부족 → classification 모델 충분히 학습 불가 → 성능 제한

? 라벨링 된 데이터가 왜 부족함?

적은 양의 라벨 데이터만으로도 다양한 자연어 태스크를 해결하고 싶다!!!



태스크 종류	설명	대표 데이터셋
텍스트 추론	문장 A가 문장 B를 의미하는지 판단 (참/거짓/중립)	SNLI, MultiNLI

태스크 종류	설명	대표 데이터셋
질문 응답	문서를 보고 질문에 대한 답을 고름	RACE, SQuAD
의미 유사도 평가	두 문장이 같은 뜻인지 판단	MRPC, STS-B
문서 분류	문장이 긍정인지 부정인지 등의 라벨을 붙임	SST-2, CoLA
상식 추론	이야기 끝부분이 자연스러운지 선택	Story Cloze Test

그래서...

“라벨이 없는 데이터로 뭔가 유용한 표현을 먼저 배워두고 → 이후 소량의 라벨로 미세 조정해야겠다!”

그런데 그걸 하려니까:

- 비라벨 데이터로 뭘 학습? (= 어떤 목표 function을 설정해야 하는가?)
- 학습한 표현을 어떻게 downstream task에 연결? (= 어떻게 transfer 할 것인가?) 라는 의문 발생



downstream task: 훈련된 모델을 통해 실제로 풀고 싶은 문제

- **Pretraining task** (사전학습에서의 임시 문제):
ex. 언어 모델한테 "다음 단어 예측해봐"
- **Downstream task** (사전학습 후 실제 목표 문제):
ex.
 - 뉴스 분류: "이 문장은 스포츠 기사야?"
 - 감정 분석: "긍정이야? 부정이야?"
 - 질의응답: "질문에 맞는 답 선택해봐."
 - 문장 유사도: "이 문장 둘 비슷해?"

⇒ 사전학습은 뇌를 키우는 과정, downstream은 뇌를 써먹는 과정

기존접근법

기존 접근법	해결 시도
모델 구조를 태스크마다 바꾸기	각 과제에 특화된 맞춤형 구조가 더 잘 맞을 거라고 판단
보조 손실 추가	태스크의 본래 목적 외에도 언어모델링, NER 등 보조 과제를 같이 학습 시켜 일반화 성능 향상 기대
LSTM, RNN 사용	문장 순서나 문맥을 잘 학습하려고 고안됨
태스크별 attention, layer 구성	문장 쌍, 질문-문맥 구조 등은 처리 방식이 달라야 하므로 아예 구조부터 다르게 설계함

태스크별 맞춤 구조, 보조 손실함수, LSTM 사용 etc...

좋은 전이 학습을 위해 매우 복잡한 구조와 맞춤형 설계가 필요

논문접근법

위와 같은 기존의 복잡한 접근과 다르게:

- 단 하나의 **Transformer만 사용**
- 입력 형식만 각 과제(task)에 맞게 변형

생성 기반 사전학습 + 식별 기반 미세조정: **2단계 학습 프레임워크** 제안

1. **사전학습**: 대용량 비라벨 텍스트 코퍼스를 기반으로 언어모델 학습
2. **미세조정(fine-tuning)**: 각 태스크의 라벨 데이터를 이용해 모델 조정

 Transformer?

2. 사전연구

2.1. NLP 반지도 학습 (Semi-Supervised Learning for NLP)

전통적 방식

- unlabeled 텍스트 데이터에서 단어 빈도나 문법적 통계를 supervised 모델의 feature로 사용
- ex. 텍스트 분류나 시퀀스 라벨링에서 n-gram 통계치나 co-occurrence를 feature로 사용



예전에는 단어나 단어 조합이 얼마나 자주 나왔는지를 숫자로 세서 feature로 씀

n-gram?

문장 속 **연속된 단어 묶음**

ex.

- 문장: "I love pizza."
- 2-gram (bigram): ["I love", "love pizza"]
- 3-gram: ["I love pizza"]

→ 이걸 등장 횟수(count)로 세서 벡터로 표현

co-occurrence?

단어끼리 **얼마나 자주 같이** 나오는가

ex.

- "love"는 "pizza"랑 자주 같이 나옴
- "hate"는 "broccoli"랑 자주 같이 나옴

→ 이걸 수치화

etc

단어 임베딩 기반 전이

- 단어 임베딩(Word2Vec, GloVe, fastText...)을 unlabeled 데이터에서 학습하고, 이를 downstream task에 활용
- but... 단어 수준의 정보만 전이 가능 = 문장/문맥 단위 이해에는 부족



word embedding?

I love pizza.

I hate broccoli.

- 컴퓨터는 love, hate, pizza, broccoli 라는 단어들이 뭘 의미인지 모름
- 그래서.. **비라벨 텍스트**들을 엄청 많이 보고 단어들 간의 **공동 등장(co-occurrence)** 관계에서 **의미 추정** 후 숫자로 표현

단어	임베딩 벡터 (예시)
love	[0.8, -0.2, 1.1, ...]
hate	[0.7, -0.3, 1.0, ...]
pizza	[0.1, 1.3, -0.4, ...]
broccoli	[0.2, 1.2, -0.5, ...]

→ 이게 unlabeled data로부터 학습한 word embedding
(love랑 hate 비슷 / pizza broccoli 비슷)

문장 수준 임베딩 시도

- 이후 문장 표현을 사전학습하는 방식(Skip-Thought, InferSent, Quick-Thought, DisSent) 등장
- 다양한 텍스트 입력을 **고차원 벡터로 변환**하여 태스크에 활용 가능, but **전이 능력의 일관성과 일반성** 면에서 한계

2.2. 비지도 사전학습 (Unsupervised Pre-Training)

일반 개념

- 초기 파라미터를 좋은 지점에서 시작, 이후 supervised 학습 성능을 높이는 전략
(원래 신경망은 파라미터(=가중치)를 랜덤으로 초기화하고 시작했는데 이걸 비지도 학습으로 뇌를 미리 키워 둔 상태에서 출발한다는 거임)
- 기존에도 이미지나 음성에서 사전학습의 효과가 입증됐었음
- 텍스트에서도 LSTM 기반 언어모델을 사전학습한 뒤, 분류 태스크에 fine-tuning 했었음
 - but 긴 문장 학습엔 제한적



LSTM은 왜 긴 문장 학습을 잘 못함?

LSTM: 순서를 따라가면서 정보를 저장하는 구조, 문장 길어질수록 앞의 정보가 점점 사라짐

- LSTM = 한 줄로 쭉 서서 귓속말 전달 게임
 - ⇒ 뒤쪽 애들은 앞에서 한 말을 잘 못 듣게 되는 상황 생김
- Transformer는 이걸 해결하려고 모든 단어가 동시에 서로를 보게 함 (self-attention)
 - 그래서 멀리 있는 단어 관계도 잘 잡아냄.

그래서...

- LSTM이 아닌 Transformer 사용: 장기 의존 문제 해결
- 단순 분류 외에도 텍스트 추론, 유사도 평가, 스토리 완성 등 다양한 과제에 높은 성능

2.3. 전이 방식 및 아키텍처 변화 (Task Transfer Strategies)

문제점

- 기존 방식들은 각 태스크마다:
 - 모델 아키텍처를 변경
 - 추가 레이어
 - 보조 손실 함수
 - 복잡한 학습 스케줄 설정
- ⇒ 일관된 모델 설계와 관리 어려움

그래서...

모델 구조는 유지하면서, 각 태스크에 맞게 입력만 변형하는 방식 고안

- ex. 문장 두 개가 들어오는 entailment 태스크는 "문장1 \$ 문장2" 형태로 입력
- ex. 질문응답 태스크는 "문맥 \$ 질문 \$ 선택지" 형태로 각각의 선택지를 모델에 통과

(1) 텍스트 추론 예시

문장1 (전제): "He is reading a book."

문장2 (가설): "He is learning something."

모델에게 이 두 문장이 논리적으로 이어지는지 물어보고 싶을 때

```
"<s> He is reading a book. $ He is learning something. <e>"
```

- `$`: 중간 구분자
- `<s>`, `<e>`: 시작/끝 표시 토큰

→ 이렇게 하나로 이어붙여서 Transformer에 넣으면 모델이 답변 가능

(2) 질문 응답 예시

문맥: "Water boils at 100 degrees Celsius."

질문: "At what temperature does water boil?"

선택지:

- (A) 100 degrees
- (B) 0 degrees
- (C) 50 degrees

```
"<s> Water boils at 100 degrees Celsius. $ At what temperature does water boil? $ 100 degrees <e>"  
"<s> Water boils at 100 degrees Celsius. $ At what temperature does water boil? $ 0 degrees <e>"  
"<s> Water boils at 100 degrees Celsius. $ At what temperature does water boil? $ 50 degrees <e>"
```

- 선택지를 추가한 모든 문장을 모델에 넣고 → 가장 높은 점수를 준 선택지를 정답으로 판단

2.4. 보조 목적 (Auxiliary Objectives)

기존 연구

- 보조 학습 목표로 언어모델링이나 다른 태스크를 함께 학습하여 성능을 높임.

본 논문

- fine-tuning 시 언어모델링 손실을 보조적으로 함께 학습함.
- 이미 사전학습 단계에서 대부분의 언어 지식을 학습 가능

⇒ 보조 손실 없이도 이미 높은 성능, 보조 손실 더하면 일부 과제에서 성능 추가 향상

3. Framework

1단계: 비지도 사전학습 (Unsupervised Pre-training)

목적

대규모 비라벨 텍스트 데이터로부터 언어 패턴 학습 → 다양한 과제에 활용 가능한 범용 표현 획득

방식

- 언어 모델링(Language Modeling): 주어진 토큰 시퀀스에서 다음 토큰 예측

앞 문맥을 보고 다음 단어를 잘 맞히면 손실이 작아지고,
틀리면 손실이 커져서 그만큼 모델을 다시 조정한다.

$$L_1(U) = \sum_i \log P(u_i \mid u_{i-k}, \dots, u_{i-1}; \Theta)$$

$$\Rightarrow L_1(U) = \sum \log P(\text{현재 단어} \mid \text{앞 단어들}; \Theta)$$

- U: 학습에 쓰인 비라벨 텍스트
- Θ : 모델의 파라미터 (가중치)
- k: 문맥 윈도우 크기
- L1: 언어 모델 손실

모델 구조

- Transformer decoder
- 여러 층 보유, 각 단어는 **Self-Attention**으로 문맥 확인

2단계: 지도 미세조정 (Supervised Fine-tuning)

목적

- 문제 풀이 (감정분석, 질문응답, 문장 추론...)
- 비지도학습된 모델을 기반으로 태스크에 맞춰 모델 미세 조정해서 문제 해결

방식

문장 입력 → 사전 학습된 GPT 모델에 통과 → 숫자 벡터 출력 → 분류기(Linear Layer) → 라벨 예측

| 입력 문장 x 를 주면, 그에 맞는 정답 y 를 맞히도록 학습

$$P(y \mid x_1, \dots, x_m) = \text{softmax}(h_m^l W_y)$$

$$L2(C) = \sum \log P(y \mid x_1, \dots, x_m)$$

$$\Rightarrow L2(C) = \sum \log P(\text{정답 라벨} \mid \text{입력 문장})$$

- $x_1, \dots, x_m$: 입력 문장
- h_m^l : 최종 레이어의 마지막 토큰 출력값
- W_y : 학습 가능한 분류 레이어 가중치
- C : 라벨이 붙은 데이터셋

3단계: 보조 학습

: 미세조정할 때 언어모델 학습을 같이 시키면 성능 향상

목적

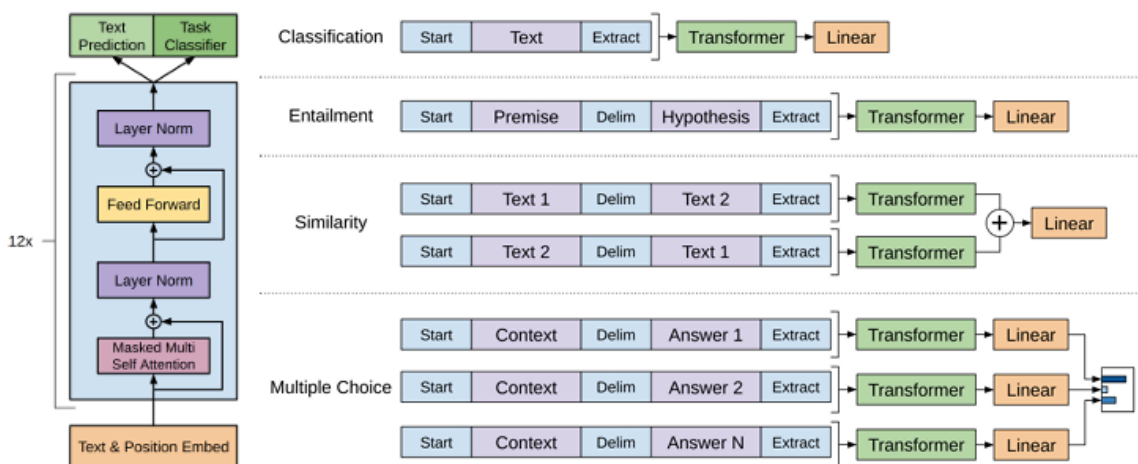
- 일반화

- 학습 속도 개선

방식

$$L3(C) = L2(C) + \lambda \cdot L1(C)$$

- λ : 조절 비율 (논문에선 0.5)



1 왼쪽: GPT 모델 내부 구조 (Transformer)

Text & Position Embed

- 텍스트 단어 + 위치 정보를 숫자(벡터)로 바꿔서 입력
 - 단어 순서도 반영

파란 박스 (12x): 12층의 Transformer 구조

- **Masked Multi-Head Self Attention:**
문장 전체를 보면서 각 단어들이 서로 어떤 관계인지 파악 → 단어 간 의미 연결
- **Layer Norm:** 계산 안정화
- **Feed Forward:** 복잡한 연산으로 표현력 향상

위 과정을 반복해서 점점 더 깊이 있는 문장 이해

초록 박스

- **Text Prediction:** 다음 단어 맞히기 → 사전학습에서 사용
- **Task Classifier:** 실제 문제 예측 (예: 긍정/부정) → 미세조정에서 사용

2 오른쪽: 태스크별 입력 구성 & 처리 방식

① Classification (분류)

- 입력: `Start + Text(문장) + Extract`
- 처리:
 - 문장을 Transformer에 넣고
 - 마지막 위치의 벡터(Extract 위치)를 꺼냄
 - Linear layer로 긍정/부정 등 라벨 예측

② Entailment (텍스트 추론)

- ex. 문장 A가 문장 B를 말하는 거임?
- 입력: `Start + 문장1(Premise) + Delim(구분자) + 문장2(Hypothesis) + Extract`
- 처리:
 - 두 문장을 하나로 붙여 넣음
 - Extract 위치의 벡터를 뽑아서 → 분류기로 "Entail", "Neutral", "Contradiction" 판단



마침표로 구분하면 되지않나? 굳이 delim이 필요?

사람은 . 보고 → 아, 문장이 나뉘는구나! 하는데

GPT는 → . 도 하나의 단어로 봄

그래서 . 이 문장 구분인지 "Mr. Smith" 에서 나오는 마침표인지 구별 못함.

③ Similarity (의미 유사도)

- 입력: 두 가지 버전 입력:
 - `Text1 + $ + Text2`
 - `Text2 + $ + Text1`
- 처리:

- 두 입력을 각각 Transformer에 넣음
- 나온 벡터 두 개를 더해서 → 하나의 벡터로 만들
- 그걸 Linear에 넣어서 유사도 점수 예측

(이중입력+합산 처리는 순서 영향을 줄이기 위함)

④ Multiple Choice (객관식)

- 예: 질문 + 여러 선택지 중 정답 고르기
- 입력:
 - 각각의 선택지마다: `Start + Context + Delim + AnswerN + Extract`
- 처리:
 - 각 선택지를 하나씩 Transformer에 넣음
 - Extract 벡터를 뽑아서 → Linear 통과
 - 모든 선택지 중 가장 높은 점수를 고름



Extract?

: GPT-1에서 “이 위치에서 출력된 벡터로 정답을 예측하자”고 지정한 **위치 토큰** 앞에 나온 문장의 요약을 뽑아낸게 extract 벡터

실제 예측(분류 등)은 오직 `<Extract>` 벡터 하나로 수행됨

왜 필요?

- 입력 문장 길이는 매번 달라짐
- 항상 같은 의미의 벡터를 뽑으려면 일관된 기준점을 정해주는 게 중요
- 그래서 항상 일정한 위치에 `<Extract>` 토큰을 붙이고, 그 자리 벡터만 사용하도록 규칙을 정한 것

어디에?

- 항상 ‘입력의 맨 마지막’에
- 왼쪽에서 오른쪽으로 문맥이 이어지는 단방향 언어모델이기 때문에, 마지막 위치의 벡터가 가장 많은 문맥 정보를 담고 있을 것이라 가정

4. Experiments

GPT-1은 사전학습 → 미세조정 프레임워크를 제안

? 정말로 사전학습을 하면 문제 푸는 성능이 좋아지나?

? 사전학습 안 한 모델이랑 비교하면 얼마나 차이 나는가?

⇒ 이걸 검증하기 위해 여러 종류의 자연어처리 과제(NLP tasks)에 GPT-1 적용

실험 과제 (Downstream tasks)

(1) 텍스트 분류 (9개)

과제명	설명	평가지표
SST-2	감정 분석 (긍정/부정)	Accuracy
CoLA	문법적 문장 판단	Matthews Corr
MRPC	문장쌍 동의 여부	Accuracy/F1
STS-B	문장쌍 유사도 점수	Pearson/Spearman corr
QQP	질문쌍 중복 여부	Accuracy/F1
MNLI	문장쌍 추론 (3-class)	Accuracy
RTE	추론 (2-class, 데이터 적음)	Accuracy
QNLI	질문 + 문장 → 정답인가?	Accuracy
WNLI	복잡한 문장 관계 추론	Accuracy

(2) 상식 추론 / 객관식 문제 (3개)

과제명	설명	특징
ROCStories	짧은 이야기, 마지막 문장 맞히기	상식적 문맥 이해
RACE	영어 독해 객관식	질문 + 문맥 + 선택지
ANLI	(추가된 실험에서) 어려운 자연어 추론	높은 난이도

입력 구성

- 모델 구조 그대로
- 문제마다 입력형식 변경:

Task Type	입력 예시
단일 문장	<Start> 문장 <Extract>
문장쌍 (추론)	<Start> 문장1 <Delim> 문장2 <Extract>
질문 + 선택지	<Start> 문맥 <Delim> 질문 <Delim> 선택지 <Extract>

⇒ 모든 문제를 하나의 텍스트 시퀀스로 변경

실험 비교 조건

실험 버전	설명
Random	사전학습 없이 처음부터 라벨 데이터로만 학습
LM Pretrained	GPT처럼 사전학습된 모델을 사용, 그 후 fine-tuning
LM Pretrained + Auxiliary	fine-tuning 할 때 언어모델 학습도 같이 (보조 손실 추가)

실험 결과

1. 사전학습을 한 모델이 훨씬 더 좋은 성능을 냄
 - 특히 라벨 데이터가 적은 문제에서 더욱 두드러짐
 - ex. RTE, STS-B, WNLI 등
2. 보조 손실 같이 사용하면 성능 더 좋아짐
 - RACE나 QNLI 같은 복잡한 문제에서 효과적
3. 구조를 바꾸지 않고도 다양한 문제에 적용 가능
 - 입력 형식만 조정하면 동일한 모델로 여러 과제를 풀 수 있음

5. Analysis

5.1 Fine-tuning 데이터 크기에 따른 성능 변화

라벨 데이터가 많으면 모델 성능이 좋아지나?

→ 사전학습 덕분에 적은 데이터로도 잘 되나?

실험 방식

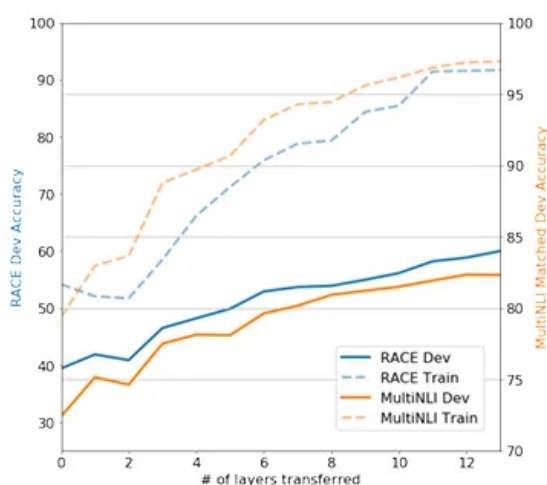
같은 과제에 대해 fine-tuning 데이터 양을 줄이면서 실험: 100%, 50%, 25%, 10% 등

결과

사전학습된 모델은 적은 데이터로도 좋은 성능

- 10%밖에 안 써도 Random Init보다 훨씬 나은 성능

1 사전학습 층 개수에 따른 비교



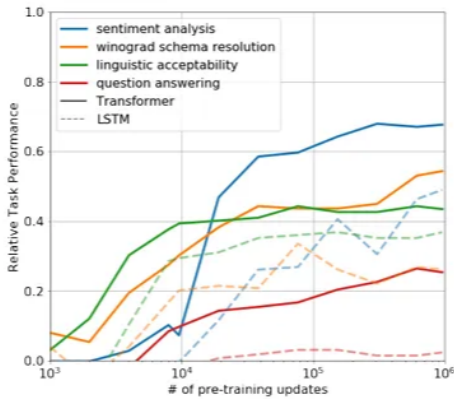
- 총 12개 층짜리 GPT 모델에서 각각 다른 개수의 층 적용
- 이후 RACE(객관식 독해)랑 MultiNLI(문장 추론)에 fine-tuning

결과

층을 더 많이 쓸수록 성능 상승

⇒ 사전학습된 층이 많을수록 모델의 언어 이해 능력 상승

2 사전학습 시간에 따른 비교



색	태스크
파랑	Sentiment analysis (감정 분류)
초록	Winograd schema resolution (상식적 지시어 판단)
주황	Linguistic acceptability (문법성 판단)
빨강	Question answering (질의 응답)
회색 점선	LSTM 모델 (각 태스크별 비교)

- 사전학습 조금 시킨 모델 / 중간 정도 학습한 모델 / 오래 학습한 모델을 다양한 과제에 적용
- 이후 fine-tuning해서 비교

결과

- 사전학습 오래 할수록 성능 상승
- Transformer(실선)이 LSTM (점선)보다 더 빠르고 많은 성능 향상

⇒ 사전학습 시간이 성능에 큰 영향

Transformer가 LSTM보다 전이학습에 더 유리

5.2 언어모델링 성능과 다운스트림 성능 간의 관계

언어모델링을 잘 하면 다른 과제도 잘 풀 수 있는가?

실험 방식

다양한 사전학습 세팅 후 언어모델 정확도(perplexity)와 downstream task 정확도 비교

결과

- 언어모델 성능이 좋을수록 다른 문제에서도 좋은 성능

⇒ GPT가 잘 예측하는 언어모델일수록 문장을 더 잘 이해하고 다른 과제에도 좋은 성능 보임

5.3 내부 attention 분석 (모델 해석)

Transformer 안에서 attention이 어떻게 작동?

실험 방식

Transformer 안의 각 attention head가 문장 내 어떤 단어에 집중하는지 시각화

결과

- 어떤 head는 문법 구조(ex. 주어-동사)에 집중
- 어떤 head는 단어 위치나 관계성(ex. "he" → "man")에 주목

⇒ GPT-10이 단순 단어 통계뿐만 아니라 문법, 구조, 의미 관계 잘 파악

yetznie@kookmin.ac.kr